

조직문화 평가 기반 실사 면담 인원 식별

설문조사 결과 기반 이직의사 예측 · 면담 인원 식별

프로젝트 배경

- 삼성전자에서는 분기별 일하는 방식(1Q,2Q) 설문조사와 연간 조직문화 진단을 실시한다.
- 진단 결과에 대한 분석은 각 팀의 CA(조직문화)가 진행하게 되는데 주어진 항목을 제외하고는 어떠한 항목이 현재 팀에 문제가 되는지 등에 대한 분석이 문항의 의도와 내용에 따라 제한됨.
- 요즘 가장 문제가 되는 이직에 대한 문항이 따로 존재하지 않으며, 별도로 조사에 의존해야 하는 부분임.
- 팀원이 300명이 넘어가는 상황에서 위와같은 조사를 단기간에 수행할수는 없음.

프로젝트 배경

- 이직의사 조사 결과를 회사 모든 구성원이 참여하는 조직문화 조사결과를 통해 유추해 내는 모델 생성 및 조직문화로 찾아내지 못하는 인원 에 대한 발견
- 기존에는 단순한 통합 점수로 팀의 위험도를 추론하지만, 모든 문항에 대한 학습을 통해 팀의 분위기를 저해하는 요인을 발견
- 도합 50문항이 넘는 조사에 대해 시각화가 매우 어렵기때문에 단순화하여 시각화 가능성 판단

WorkFlow

사용자 전처리부



Name	Number	Date
Name	120	2024/10
John	123	2024/10
Sehney	23	2024/11
Baboma	1,3	2024/12
John	.1	2024/09

CSV로 변환해야하는 항목

1. SCI 진단 결과
2. 1Q 일하는 방식 조사
3. 2Q 일하는 방식 조사

프로젝트 동작 범위



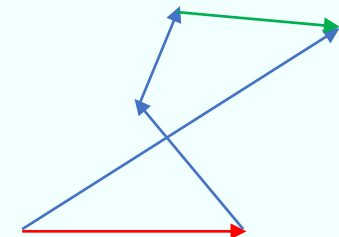
RandomForest
Classification

Feature

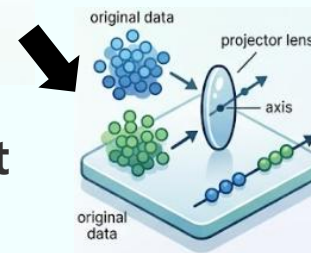
- 각 조사 결과
- 증감 값
- Identity
 - 소속
 - 직급
 - 학력

Target

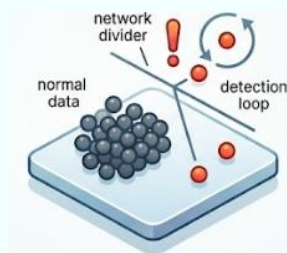
- 이직 의사



Major feature
추출



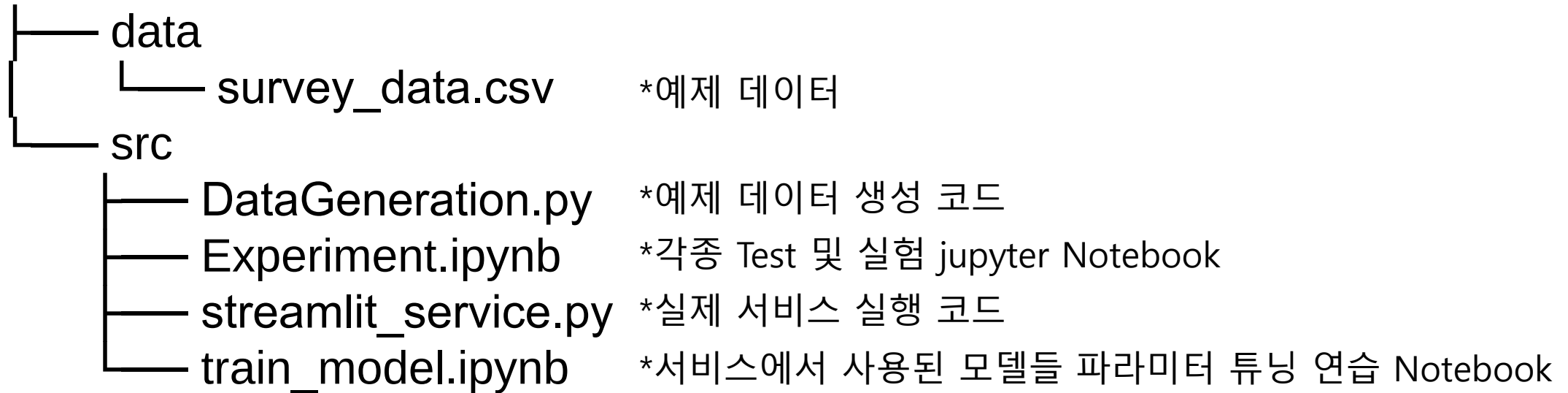
LDA 차원 축소



Isolation Forest
이상치 탐색

면담
명단

디렉토리 설명



데이터 소개



인사정보(대외비)로써 반출불가
생성형 AI에게 로직을 전달하여 생성

- 직급 : CL1~CL4, 실제 부서내 비율에 맞게 조정
- 소속 : 8개의 그룹(A~H) 데이터 생성시는 균등 분배
- 학력 : 고졸(CL1), 학사, 석사, 박사, 실제 비율에 맞게 조정
- 근속년수 : 1~40까지 랜덤 배정
- 조직문화 평가
 - 복지 : 회사에서 제공하는 복지가 만족스러운지
 - 회의 : 회의의 빈도 및 만족도, 후속 회의 여부 등. 회의 문화 점검
 - 자기계발(교육) : 자기계발에 힘쓸 환경이 조성되어있는지, 만족하는지
 - 업무 : 현재 담당하고 있는 업무에 대한 만족도
 - 상사 : 상사에게 피드백은 받고있는지, 불화는 없는지
 - 부서만족도 : 현재 소속하는 부서의 방향성 및 비전
 - C레벨 : CEO, CTO등 임원에 대한 만족도
 - 동료 : 친하게 지내는 동료 및 같이 일하는 동료의 만족도
 - 전년 대비 증감 : 작년대비 조직문화 평가 값의 증감
- 업무 만족도
 - 고객 : 고객 위주로의 업무가 이루어지고있는지
 - 회의 : 회의 문화에 만족하는지
 - 기술력 : 기술력 확보의 방향으로 업무가 이루어지고있는지
 - 소통 : 동료들과 소통 및 문제 드러내기에 적극적인지
 - 보고체계 : 보고체계가 간소화 되어있는지

데이터 생성 코드

```
def _generate_tenure(positions, rng):
    """직급이 높을수록 근속연수가 길어지도록 상관관계를 부여하여 근속연수(정수)를 생성."""
    position_base_years = {"CL1": 2, "CL2": 6, "CL3": 12, "CL4": 20}
    base = np.array([position_base_years[p] for p in positions])
    years = np.clip(rng.normal(loc=base, scale=4), 0, 35)
    return np.round(years).astype(int)
```

```
def _generate_positions(n, rng):
    return rng.choice(POSITIONS, size=n, p=POSITION_PROB)
```

```
def _generate_groups(n, rng):
    return rng.choice(GROUPS, size=n)
```

```
def _generate_education(n, rng):
    return rng.choice(EDUCATIONS, size=n, p=EDUCATION_PROB)
```

```
POSITIONS = ["CL1", "CL2", "CL3", "CL4"]
POSITION_PROB = [0.05, 0.25, 0.30, 0.40]
```

```
GROUPS = [f"{c}그룹" for c in "ABCDEFGH"]
```

```
EDUCATIONS = ["고졸", "학사", "석사", "박사"]
EDUCATION_PROB = [0.05, 0.45, 0.35, 0.15]
```

- 실제 데이터 베이스 기반 생성

데이터 생성 코드

```
def _generate_likert_block(columns, n, rng, base_corr=0.35, extra_corr=None):
    """
    공통 잠재요인 + 항목별 노이즈로 상관관계를 가진 1~5점 리커트 응답을 생성.
    extra_corr: {(col_a, col_b): corr} 형태로 특정 항목 쌍에 추가 상관관계를 부여할 때 사용.
    (예: README에 언급된 상사-업무 만족도 간 다중공선성 재현)
    """
    n_items = len(columns)
    corr = np.full((n_items, n_items), base_corr)
    np.fill_diagonal(corr, 1.0)

    if extra_corr:
        for (col_a, col_b), value in extra_corr.items():
            i, j = columns.index(col_a), columns.index(col_b)
            corr[i, j] = corr[j, i] = value

    latent = rng.multivariate_normal(mean=np.zeros(n_items), cov=corr, size=n)
    scores = np.clip(np.round(latent * 1.1 + 3.3), 1, 5).astype(int)
    return pd.DataFrame(scores, columns=columns).astype("Int64")
```

```
def _apply_missing_responses(df, columns, response_rate, rng):
    """
    설문(prefix) 단위로 응답 여부를 결정한다.
    응답하지 않은 사람은 해당 질문에 속한 문항 전체가 결측(NA) 처리된다.
    response_rate는 스칼라 또는 사람별 응답률 배열(예: 위험군만 낮은 배{
    """
    n = len(df)
    responded = rng.random(n) < response_rate
    df.loc[~responded, columns] = pd.NA
    return df
```

- 점수 결과는 정규분포 기준 난수이지만, 어느정도 feature간의 영향성을 반영하도록 작성
- 항상 100% 응답이 아닌, 설문마다 평균적 응답률을 적용하여 결측치 생성
 - 결측치는 극단적 성향(이직 의사 높음)에서 좀더 높게 반영되도록 생성

데이터 생성 코드

```
def _zscore(s):
    return (s - s.mean()) / s.std(ddof=0)

volatility = raw_100_scores.std(axis=1, ddof=0)
mean_score = raw_100_scores.mean(axis=1)

risk_score = _zscore(volatility) - _zscore(mean_score)
risk_score = risk_score + rng.normal(0, noise_std, size=len(raw_100_scores))

tier5_cutoff = risk_score.quantile(1 - tier5_rate)
tier4_cutoff = risk_score.quantile(1 - tier4_rate)

is_tier5 = risk_score >= tier5_cutoff
is_tier4 = (risk_score >= tier4_cutoff) & ~is_tier5
is_remaining = ~(is_tier5 | is_tier4)

levels = pd.Series(0, index=risk_score.index, dtype=int)
levels[is_tier5] = 5
levels[is_tier4] = 4

# 정규분포 가정: 나머지 70%를 재표준화한 뒤, 정규분포 3분위 경계( $\pm \text{norm.ppf}(2/3)$ )로 1~3점 배분
remaining_z = _zscore(risk_score[is_remaining])
band = norm.ppf(2 / 3)

levels[is_remaining & (remaining_z > band)] = 1
levels[is_remaining & (remaining_z <= band) & (remaining_z > -band)] = 2
levels[is_remaining & (remaining_z <= -band)] = 3
```

• Target 생성시

이상치 생성을 위하여 전체적으로 점수가 낮으면 이직의사가 높음으로 표시하였지만, 좀더 극단적인 성향을 띌수 있도록 생성(4~5점)

- 1~3점은 회사 분위기에 따라 생기는 것처럼 정규 분포를 따를수 있도록 랜덤 배치

※세부 내용은 코드 참조

데이터 생성 코드

```
# 근속년수 → 4구간(신입/주니어/미들/시니어)
I_fresh = ((df["근속년수"] >= 0) & (df["근속년수"] < 3)).astype(int)
I_junior = ((df["근속년수"] >= 3) & (df["근속년수"] < 10)).astype(int)
I_middle = ((df["근속년수"] >= 10) & (df["근속년수"] < 20)).astype(int)
I_senior = (df["근속년수"] >= 20).astype(int)
df["근속년수_구분"] = 1 * I_fresh + 2 * I_junior + 3 * I_middle + 4 * I_senior

df["직급"] = df["직급"].map(CL_MAP)
df["학력"] = df["학력"].map(DEGREE_MAP)
df[["직급", "학력"]] = df[["직급", "학력"]].fillna(0)
```

Data들중에 string 계열인 값들은 학습을 위하여 지시함수(근속년수 -> Class분류) 혹은 int로 변환

⇒ 후술하겠지만 숫자가 높아지는것이 모델 학습시 반영되는 경향이 있으므로 위에 기술된 Feature들처럼 숫자가 높은것과 실제 의미가 높아지는 경우에만 위와같은 변환을 수행하도록 함

프로젝트 코드 및 내용 기술

```
df = pd.get_dummies(df, columns=["소속"], drop_first=False)
# 업로드 데이터에 8개 그룹이 모두 없을 수 있으므로 누락 더미는 False로 채운다.
for col in GROUP_ITER:
    if col not in df.columns:
        df[col] = False
```

그룹의 경우는 학습시 1~8로 변환하면
그룹 Feature의 값이 높을수록 이직의사 Target의 값에 영향을
줄수 있으므로 One-hot encoding이 더 유효

프로젝트 코드 및 내용 기술

```
df_selection["회의"] = df["1Q_업무몰입도평가_회의"] + df["2Q_업무몰입도평가_회의"]
df_selection["업무"] = (df["1Q_업무몰입도평가_고객"] + df["2Q_업무몰입도평가_고객"]
|   |   |   |   |   | + df["1Q_업무몰입도평가_기술력"] + df["2Q_업무몰입도평가_기술력"])
df_selection["상사"] = (df["1Q_업무몰입도평가_보고"] + df["2Q_업무몰입도평가_보고"]
|   |   |   |   |   | + df["1Q_업무몰입도평가_소통"] + df["2Q_업무몰입도평가_소통"])
```

실제로 다중공선성이 있는 feature들에 대해서는 전부 점수를 합하여 Target에 대한 영향력은 갖추되, Feature의 값 자체는 1~5로 무시하기에는 무의미하게 큰 값들은 아니기때문에 합친뒤 추후 Standardization을 통해 사용

프로젝트 코드 및 내용 기술

```
rfc = RandomForestClassifier(random_state=0)
rfc.fit(X_te_std, y_te)

importance = (pd.Series(rfc.feature_importances_, index=FEATURES)
              | .sort_values(ascending=False))
top3 = importance.head(3)

st.markdown("**Major Feature 컬럼:** " + ", ".join(f"`{c}`" for c in top3.index))
```

RandomForest를 통해 앞서 정리된 Feature들의
Standardization을 통해 모델 학습

RandomForest 선택 이유

1. Voting(SVC, NaiveBayes, Logistic Regression),
DecisionTree, RandomForest 중 하이퍼 파라미터 튜닝
결과에서 가장 유의미한 값 선택
2. Feature_importance를 통해 분류할수 있는 Model

```
=====RandomForest=====
precision    recall  f1-score   support

0           0.61      0.50      0.55         44
1           0.71      0.80      0.75         69

accuracy          0.68         113
macro avg          0.66      0.65      0.65         113
weighted avg          0.67      0.68      0.67         113
```

```
=====Voting=====
precision    recall  f1-score   support

0           0.66      0.48      0.55         44
1           0.72      0.84      0.77         69

accuracy          0.70         113
macro avg          0.69      0.66      0.66         113
weighted avg          0.69      0.70      0.69         113
```

```
=====tree=====
precision    recall  f1-score   support

0           0.58      0.16      0.25         44
...
accuracy          0.63         113
macro avg          0.61      0.54      0.50         113
weighted avg          0.61      0.63      0.56         113
```

참고(실험 코드)

```
tree_model = DecisionTreeClassifier(random_state=0,max_features=3,max_depth=3)
```

```
logistic = LogisticRegression()
```

```
svm = SVC(kernel='linear',C=1, degree=4)
```

```
GNB = GaussianNB()
```

```
voting = VotingClassifier(
```

```
    estimators=[
```

```
        ('lm',logistic),
```

```
        ('gnb',GNB),
```

```
        ('svm',svm)
```

```
    ],
```

```
    voting='hard',
```

```
    weights=[1,1,1]
```

```
)
```

```
rfc = RandomForestClassifier()
```

프로젝트 코드 및 내용 기술

```
lda = LinearDiscriminantAnalysis(n_components=2)
lda.fit(X_te_std, y_te)
X_lda = lda.transform(X_std)

lda_df = pd.DataFrame(X_lda, columns=["lda_comp1", "lda_comp2"], index=X.index)
lda_df['target'] = df_raw.loc[X.index, '이직의사'].values
```

LinearDiscriminantAnalysis를 통한 차원 축소

처음에는 통합점수에 의한 분석보다 유의미한 결과를 도출할 수 있으나 확인을 위한 실험이었으나, 중앙값을 통한 비교 결과 유의미한 해석은 불가능함.

최종적으로 이상치 탐색 후 시각화를 위한 차원 축소의 도구로서만 활용
세부 실험 및 결론 도출은 다음장 참조

차원 축소 실험

- PCA(Principal Component Analysis)

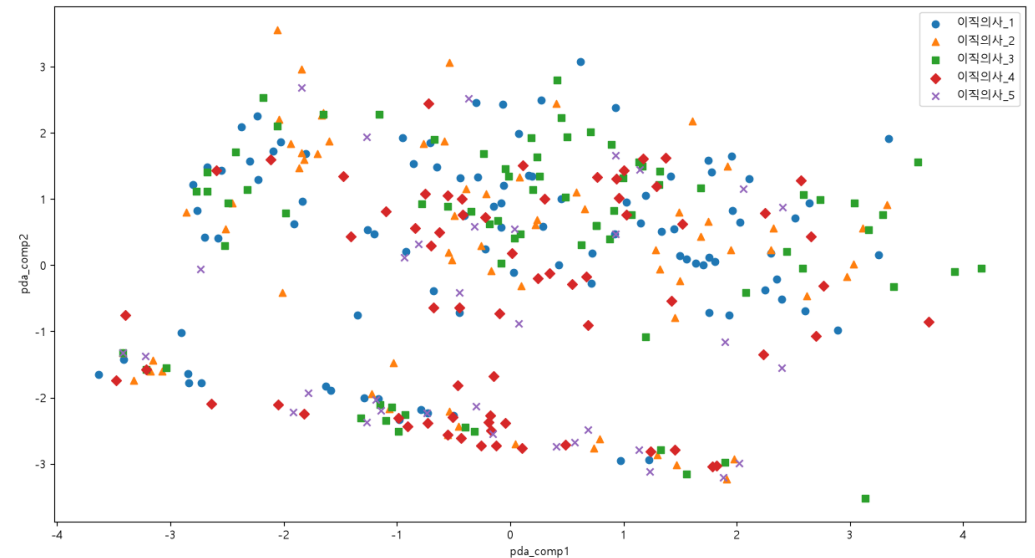
Target이 없는 비지도 학습으로 Feature들의 분산을 최소화 하는 방향만 고려되기때문에 실 Data로는 다를수 있으나, 현재 DataGeneration상으로는 유의미한 결과를 보기는 어렵다고 판단

```
import matplotlib.pyplot as plt
```

```
pca_idintiy = PCA(n_components=1)
pca_idintiy.fit(X_tn_std)
```

```
X_tn_identity_pca = pca_idintiy.transform(X_tn_std)
X_tn_pca_df['pca_comp2'] = X_tn_identity_pca
```

```
X_tn_pca_df['target'] = df_group.loc[X_tn.index, '소속'].values
X_tn_pca_df
```



차원 축소 실험

- LDA 1차원 & 1차원 합성

설문 결과 Feature로 Target을 학습하는 상황에서 1개의 차원과

Identity(소속, 학력, 직급, 근속년수) Feature로 Target을 학습하는 1개의 차원으로 2차원으로 축소

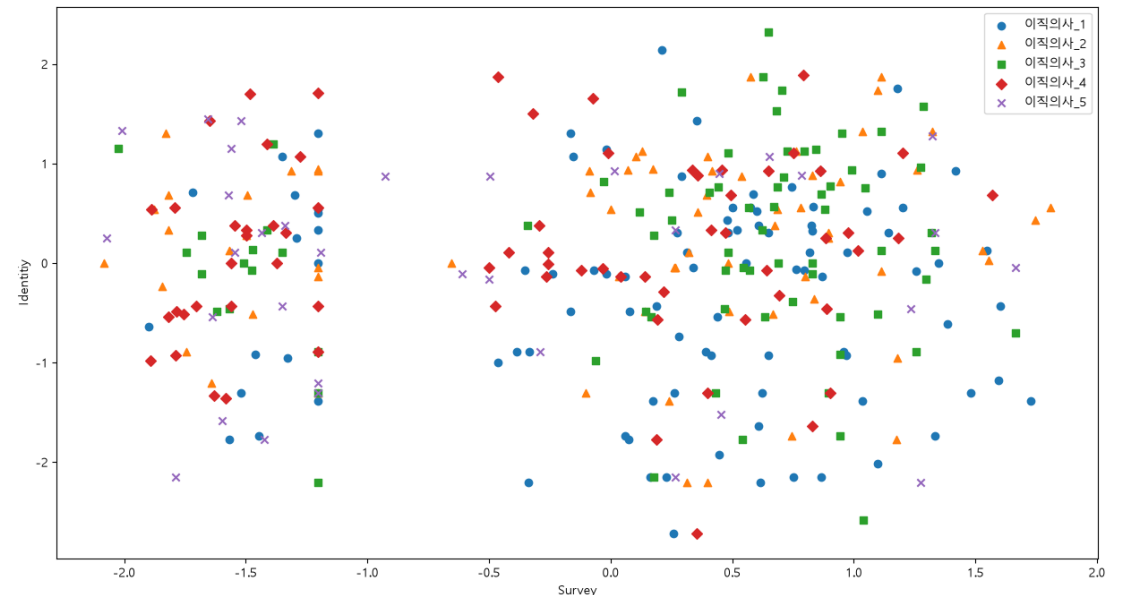
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X_survey = df_selection[TRAIN_FEATURE_SURVEY]
X_identity = df_selection[TRAIN_FEATURE_IDENTITY]
y = df['이직의사']
lda = LinearDiscriminantAnalysis(n_components=1)

X_tn_sur, X_te_sur, y_tn_sur, y_te_sur = train_test_split(X_survey, y, random_state=0, stratify=y)
X_tn_id, X_te_id, y_tn_id, y_te_id = train_test_split(X_identity, y, random_state=0, stratify=y)

lda.fit(X_tn_sur, y_tn_sur)
X_tn_lda_sur = lda.transform(X_tn_sur)
X_te_lda_sur = lda.transform(X_te_sur)

lda.fit(X_tn_id, y_tn_id)
X_tn_lda_id = lda.transform(X_tn_id)
X_te_lda_id = lda.transform(X_te_id)
```



차원 축소 실험

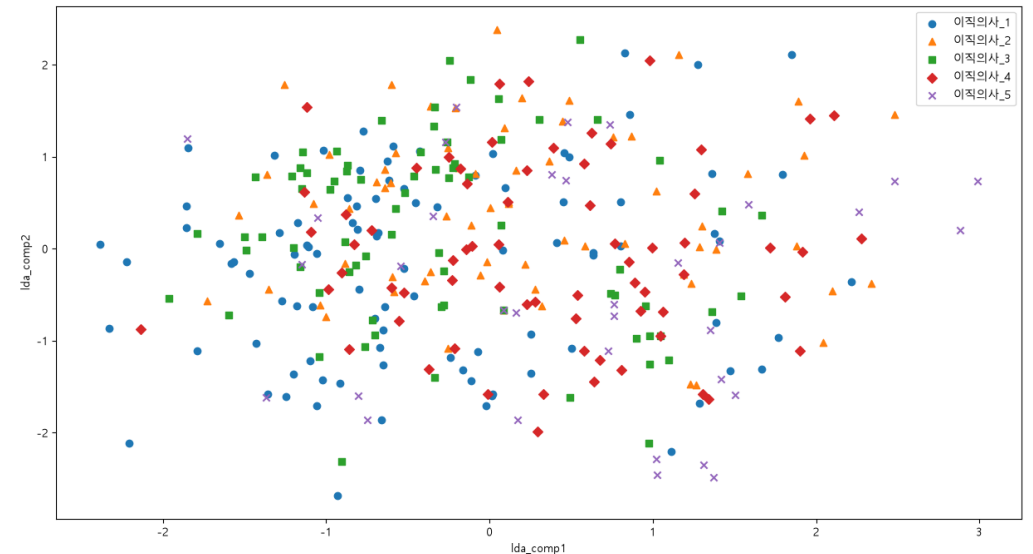
- 모든 Feature를 통한 LDA 2차원 축소
- 가장 고르고 유의미해보이나, 실제 중간값 비교를 하여 시각화 해본결과 의도된 결과는 아니나 유의미한 결과 도출

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

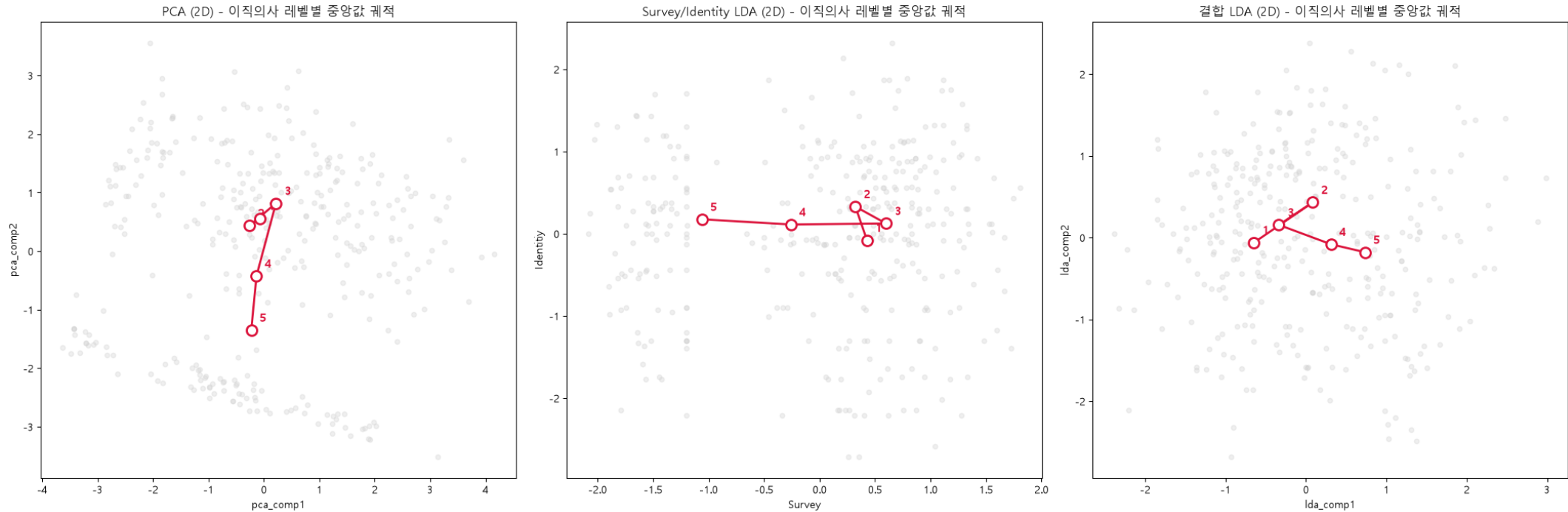
plt.figure(figsize=(15,8))

lda = LinearDiscriminantAnalysis(n_components=2)
lda.fit(X_tn_std, y_tn)
X_tn_lda_combined = lda.transform(X_tn_std)

X_tn_lda_combined_df = pd.DataFrame(X_tn_lda_combined, columns=['lda_comp1', 'lda_comp2'])
X_tn_lda_combined_df['target'] = y_tn.values
```



차원 축소 실험 결론



AI에게 각 Target별 중앙값으로 변위를 시각화 해달라고 요청한 결과
PCA와 LDA 1차원 합성의 결과는 전혀 해석할수 없는 결과를 보임.

LDA 2차원의 경우, 1~3은 정규분포로 배치, 4,5는 극단의 비율을 보이도록 요청하였기때문에 DataGeneration의 알고리즘을 일부 반영된것으로 보임.

차원 축소 결론(AI 분석 결과)

3. 교란요인

4·5의 분리에는 "태도가 낮다"뿐 아니라 ****응답을 덜 해서 0이 많다(결측률 0.38 / 0.49)****가 크게 기여한다. 이는 생성 설계상 의도된 상관(미응답 ↔ 위험도)이라 정당한 신호지만, 분리 방향이 부분적으로 ****응답 성실도(결측 패턴)****를 인코딩한다는 점을 함께 밝혀야 해석이 정직하다.

4. 결론

데이터 생성 알고리즘상 이직의사는 연속 척도가 아니라 정규분포 몸통(1~3점)과 극단 꼬리(상위 30%인 4~5점)로 설계되었고, 신원 속성은 이직의사와 독립적으로 생성되었다. 이 구조는 세 임베딩 결과로 그대로 확인된다. ****(1)**** Survey/Identity LDA는 이직의사와 무관한 신원 축이 노이즈로 작용해 유의미한 분리를 만들지 못했다. ****(2)**** PCA는 비지도 축소임에도 최대 분산 방향이 고위험군의 저점수·고결측 패턴과 일치해 4~5점을 분리했다. ****(3)**** 결합 LDA는 지도 학습으로 몸통과 꼬리를 떼어내는 방향을 명시적으로 찾아, 고위험군을 가리키는 방향성 측면에서 가장 해석 가능한 축을 제공했다. 다만 어느 임베딩도 1~5점의 선형적 순서는 복원하지 못했는데, 이는 데이터가 선형 연속체가 아니라 몸통-꼬리 구조로 설계된 것과 일치한다.

프로젝트 코드 및 내용 기술

```
iso = IsolationForest(n_estimators=estimator, contamination=contamination, random_state=0)
iso.fit(X_te_std)
iso_predict = iso.predict(X_std)
```

```
outlier_mask = iso_predict == -1
high_risk_mask = lda_df['target'] == 4
high_risk_outlier_mask = high_risk_mask & outlier_mask
```

```
ext_high_risk_mask = lda_df['target'] == 5
ext_high_risk_outlier_mask = ext_high_risk_mask & outlier_mask
```

IsolationForest를 통한 이상치 탐색 수행 및 고위험군에 대한 masking

IsolationForest로 탐색한 이유

1. OneClass SVM은 단일 Feature에 대한 이상치 탐색이므로 Feature를 반영한 Target의 이상치 탐색 불가
2. Local Outlier Factor의 경우 근접한 데이터를 보기때문에 결측치나 상관관계가 강하게 드러나는 현 상황에서 적용시 이상치 감지율이 매우 낮음

UI/UX 설명

↪ 사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files



survey_data.csv
36.9KB



CSV 업로드 완료

📊 위험 요소 분석

👁️ 이직 고위험군 탐색

조직문화 평가 기반 위험군 분류

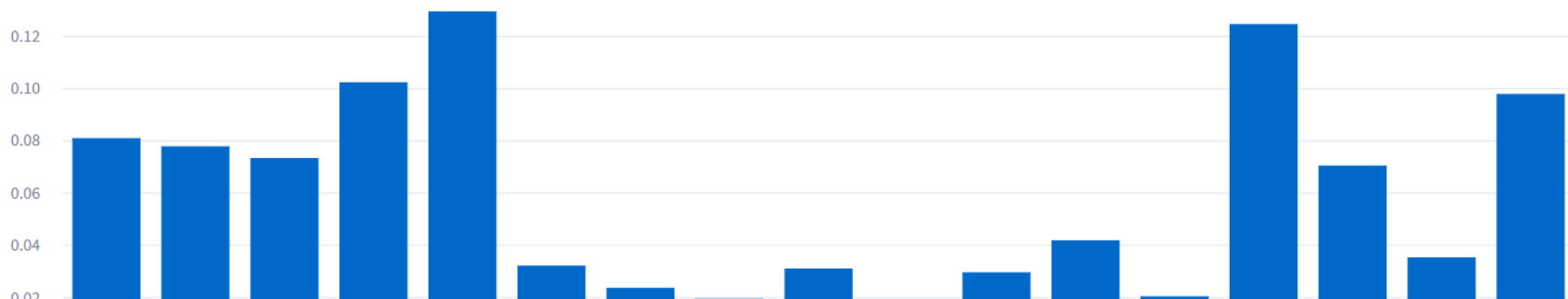
RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

위험 요소 분석

RandomForest 기반 이직 결정 요소 Top 3

Major Feature 컬럼: 상사, 업무, 복지

전체 Feature Importance



UI/UX 설명

↪ 사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files



survey_data.csv
36.9KB



CSV 업로드 완료



위험 요소 분석



이직 고위험군 탐색

조직문화 평가 기반 위험군 분류

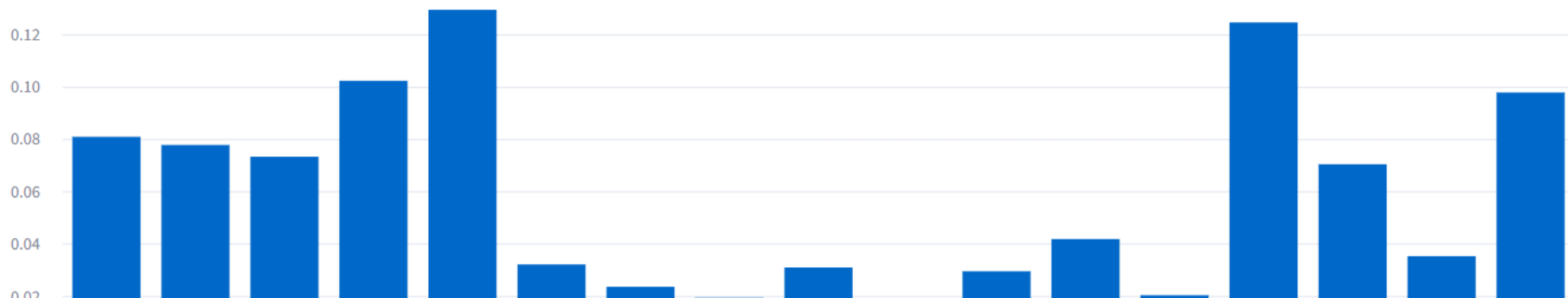
RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

위험 요소 분석

RandomForest 기반 이직 결정 요소 Top 3

Major Feature 컬럼: 상사, 업무, 복지

전체 Feature Importance



UI/UX 설명

↪ 사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files

survey_data.csv
36.9KB

CSV 업로드 완료

📊 위험 요소 분석

👁️ 이직 고위험군 탐색

.CSV 파일 업로드를 통해 데이터 전달

조직문화 평가 기반 위험군 분류

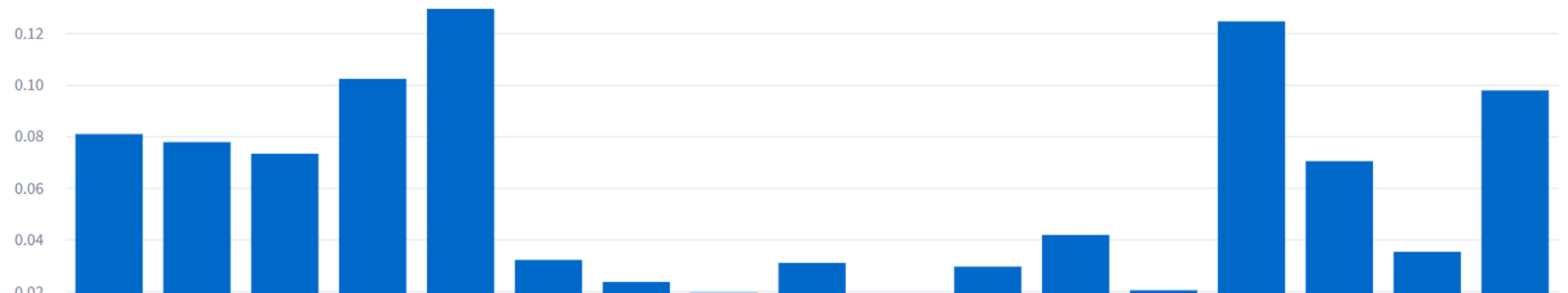
RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

위험 요소 분석

RandomForest 기반 이직 결정 요소 Top 3

Major Feature 컬럼: 상사, 업무, 복지

전체 Feature Importance



UI/UX 설명

↪ 사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files



survey_data.csv
36.9KB

CSV 업로드 완료

위험 요소 분석

이직 고위험군 탐색

조직문화 평가 기반 위험군 분류

RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

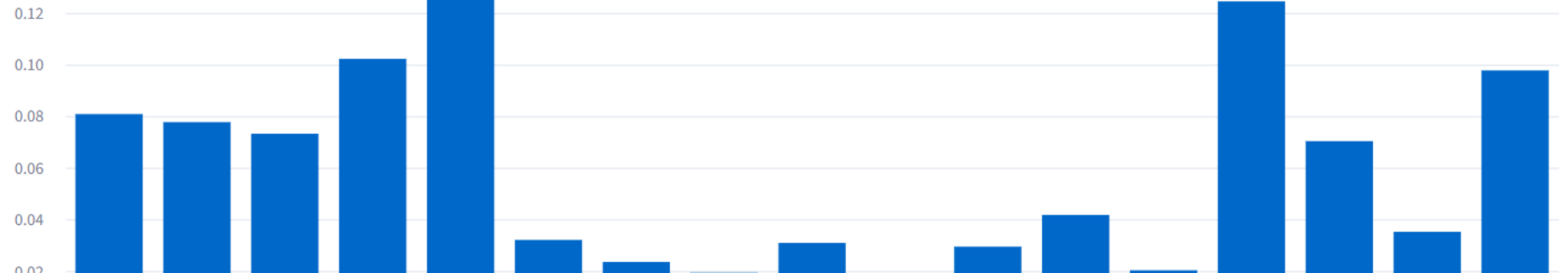
위험 요소 분석

RandomForest 기반 이직 결정 요소 Top 3

Major Feature 컬럼: 상사, 업무, 복지

주요 기능 2가지 선택 sidebar 구성

전체 Feature Importance



UI/UX 설명

↪ 사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files



survey_data.csv
36.9KB



CSV 업로드 완료



위험 요소 분석



이직 고위험군 탐색

조직문화 평가 기반 위험군 분류

RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

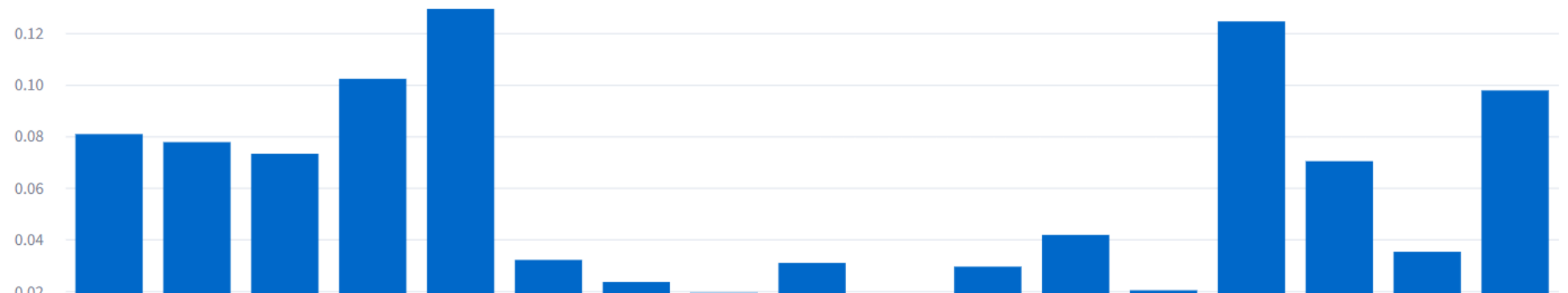
위험 요소 분석

RandomForest 기반 이직 결정 요소 Top 3

Major Feature 컬럼: 상사, 업무, 복지

Data에 따라 RandomForest 수행결과에 따른
Major Feature 출력

전체 Feature Importance



UI/UX 설명

↪ 사이드바 메뉴

📁 설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files



survey_data.csv
36.9KB



CSV 업로드 완료

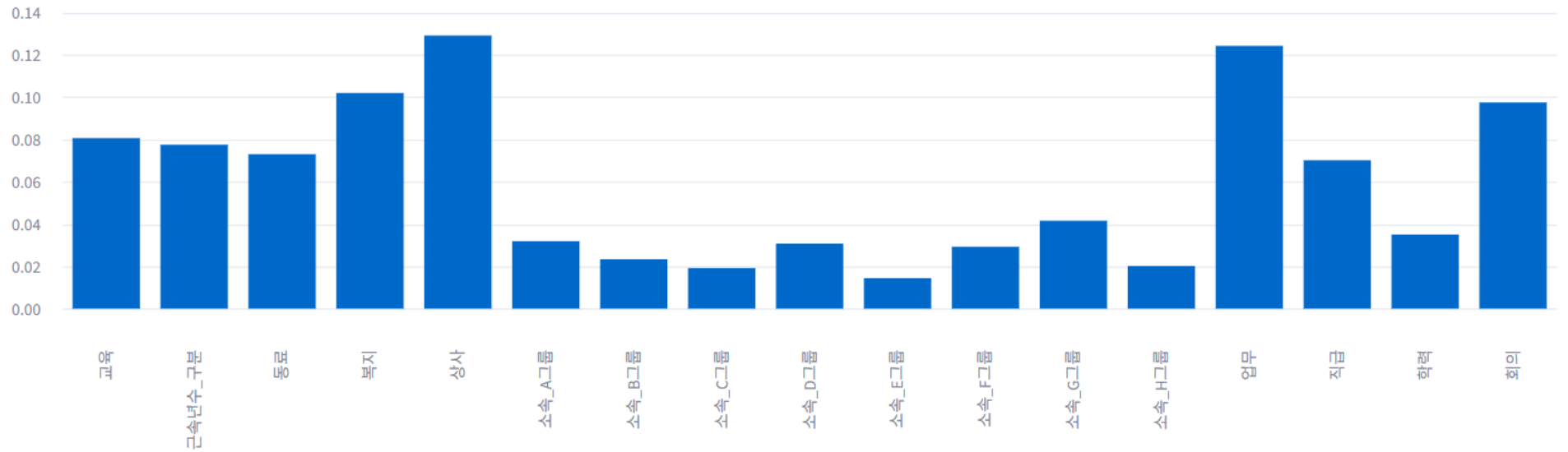


위험 요소 분석



이직 고위험군 탐색

전체 Feature Importance



전체 Feature에 대한 importance값 출력

feature	importance
교육	0.0806
근속년수_구분	0.0775
동료	0.073
직급	0.0701
소속_G그룹	0.0416
학력	0.035

UI/UX 설명

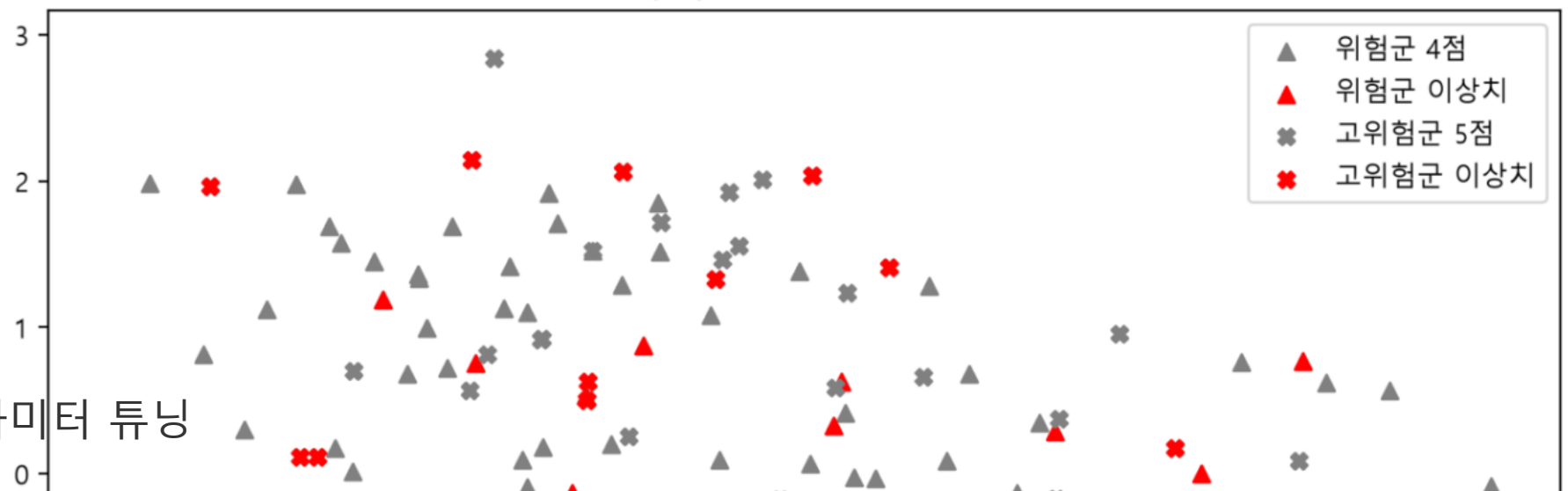
조직문화 평가 기반 위험군 분류

RandomForest기반의 주요 Feature분석 / LDA를 통한 데이터 시각화 / Isolation Forest 기반의 이상치 탐지

위험 요소 분석

LDA(2D) 공간상의 Isolation Forest 고위험군 시각화

결합 LDA (2D) - Isolation Forest 고위험군



Browse files

survey_data.csv
36.9KB

CSV 업로드 완료

위험 요소 분석

이직 고위험군 탐색

Isolation Forest estimator

100

Isolation Forest contamination

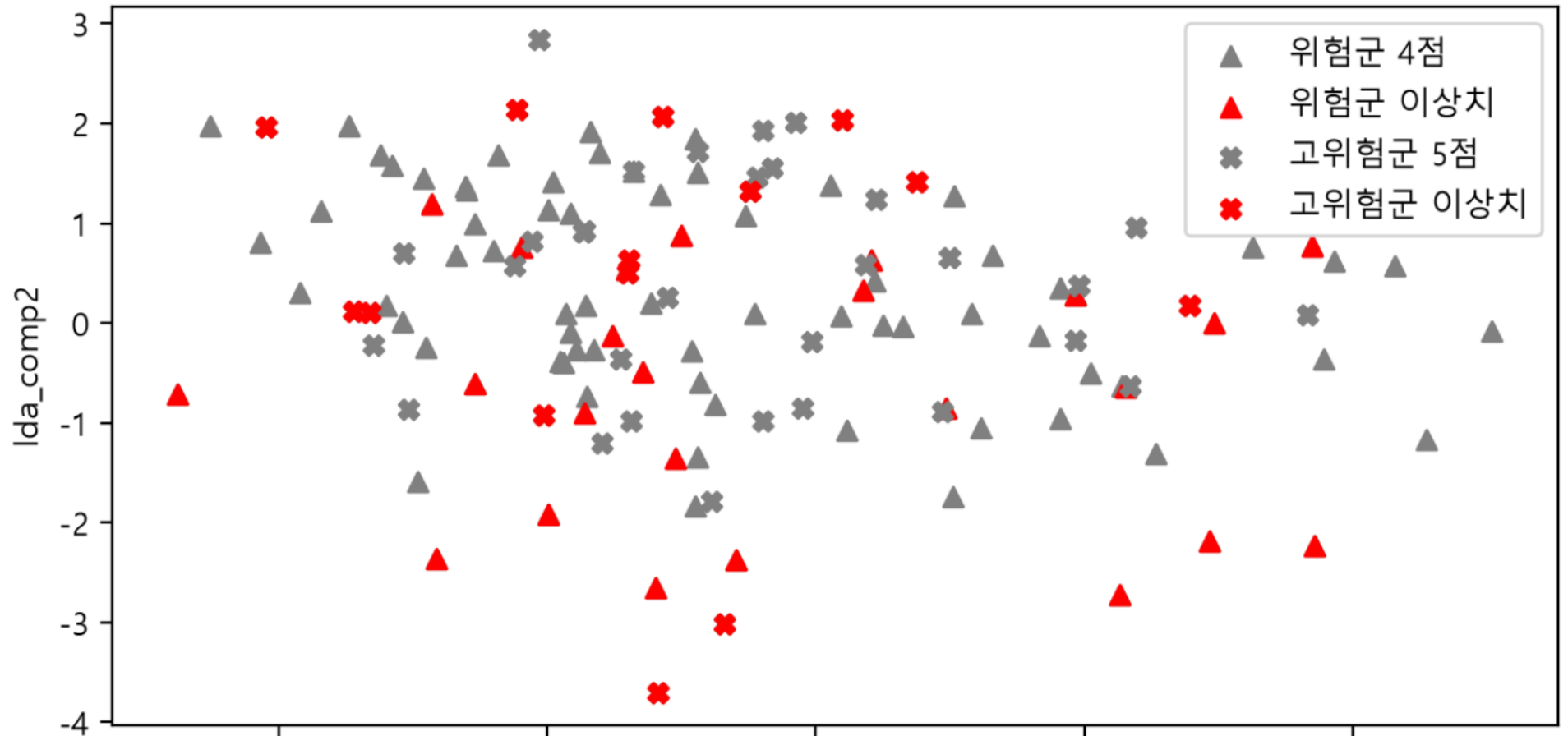
auto

Isolation Forest 하이퍼 파라미터 튜닝
슬라이드바

UI/UX 설명

위험군 고위험군의 분포와 이상치 시각화

LDA(2D) 공간상의 Isolation Forest 고위험군 시각화



사이드바 메뉴

설문 CSV 업로드

Drag and drop file here

Limit 200MB per file • CSV

Browse files

survey_data...
36.9KB

CSV 업로드 완료

위험 요소 분석

이직 고위험군 탐색

Isolation Forest estimator

100

Isolation Forest contamination

auto

UI/UX 설명

Limit 200MB per file • CSV

Browse files



survey_data...
36.9KB

CSV 업로드 완료



위험 요소 분석



이직 고위험군 탐색

Isolation Forest estimator

100

Isolation Forest contamination

auto

고위험군 인원: 142명 / 전체 450명 (31.6%)

고위험군 명단

행번호	소속	직급	학력
27	C그룹	CL1	학사
33	F그룹	CL3	학사
50	A그룹	CL2	고졸
53	D그룹	CL4	학사
79	E그룹	CL4	석사
80	E그룹	CL4	학사
81	F그룹	CL3	학사
96	E그룹	CL3	석사
100	G그룹	CL4	박사
103	H그룹	CL4	석사

면담 필요 인력 (isolationForest 이상치 & 위험군(4점 이상)) 명단 출력

프로젝트 결론

• 프로젝트 성과

- “종합 점수” 분석의 위험성, 어떠한 방법의 차원축소도 유의미한 축소를 이뤄내기 어려움
- 이상치 탐지의 효과, 정량적으로 예측하기 어려운 데이터, 특히 인사 데이터들엔 꽤 유효한 분석 도구

• 모델 · 분석 결과

- RandomForest : Feature가 많고 질문에 답변하는 식의 Feature에서 매우 유효한 결과를 가져옴. 다만 전부다 동일한 답변으로 선택했을때 맞출확률인 25% 보단 높아야 유효하지않을까 생각함
- 결합 LDA 고위험군 분리 : 앞서 기술한것처럼 실제 데이터에선 다를수 있지만 LDA에 의한 차원축소가 그나마 효과적임
- Isolation Forest 면담 명단 : 이상치 분석은 잘못된 데이터의 신뢰도를 올리는 방법중 하나라고 생각했지만 본 프로젝트의 시발점인 잘못된 데이터를 탐지해야하는 상황에선 매우 유용하게 쓰일수 있었음.

• 한계 및 교훈

- 결국 데이터의 양이 많아지고 예측 모델에 맞지않을정도로 큰 사건에 의해서 팀이 분열하는 경우는 예측이 모두 빗나갈 가능성이 높다. 사람의 감정이 반영될수 있기 때문이다.
- 그룹원의 숫자가 적은곳은 결측치가 아무리 적어도 뚜렷한 결과를 뽑지는 못할수 있다.
- 결국 직접 면담을 해야 이상치에 대한 검증이 가능할것이며, 또한 본 모델로 인해 놓치는 중요한 의견도 분명 있을것이다. 예측 Accuracy가 엄청 높은 편은 아니기 때문이다.

• 향후 활용 · 개선

- 이상치들을 모아서 이상치들에 대한 학습을 시도해보고 조직을 나가고싶은 사람들에게 대한 근원적 불만 요소를 탐지
- 차원 축소를 2차원이 아닌 3차원 상에서 시도해보고 그때는 유의미한 데이터를 추론해낼수 있지않을까 재실험 해본다.